

A Deep Learning-Based Experiment on Forest Wildfire Detection in Machine Vision Course

1. Introduction

Wildfires are one of the most destructive natural disasters, with their frequency and intensity increasing due to climate change. The year 2023 witnessed devastating wildfires in Canada, leading to severe environmental damage, loss of property, and threats to human lives. Early detection and response are crucial to mitigating the damage caused by wildfires. However, traditional methods of fire detection, such as satellite imaging and manual observation, are often limited by delays and inaccuracies.

This project aims to leverage the power of deep learning to automatically detect wildfires using images. The solution proposed involves building a Convolutional Neural Network (CNN) based on a modified VGG16 architecture, designed to identify fire presence in real-time. This model can potentially be integrated into automated surveillance systems for rapid detection and response.

Traditional fire detection systems, such as satellite monitoring and human observation, often suffer from delays in response time due to limitations in spatial and temporal resolution. As a result, there is a critical need for automated systems that can detect wildfires quickly and accurately to prevent widespread damage.

Motivation

The motivation behind this project is to leverage deep learning technologies to develop an automated wildfire detection system using computer vision. By utilizing Convolutional Neural Networks (CNNs), specifically a modified VGG16 architecture, this project aims to identify fires from images captured in real-time by surveillance cameras or drones. This approach can potentially lead to faster detection and allow authorities to respond promptly to emerging fires.

Objectives

The key objectives of this project are as follows:

1. Develop a deep learning model capable of distinguishing between fire and non-fire images.
2. Minimize false negatives to ensure all actual fires are detected.
3. Test the model's robustness in varying environmental conditions.
4. Assess the potential for deploying this system in real-time applications.

2. Problem Statement

Challenges in Wildfire Detection and Limitations of Traditional Methods

The primary challenge in wildfire detection is **accurately identifying fires from images**, especially in varying lighting conditions, diverse landscapes, and changing weather patterns. Misclassification in this context can lead to two critical errors:

- **False Positives (Type I Errors):** These occur when the system incorrectly identifies non-fire scenarios—such as sunsets or city lights—as fires, which can lead to unnecessary alarms and waste resources.
- **False Negatives (Type II Errors):** These are more concerning, as they involve failing to detect actual fires, which can delay response times and result in greater damage. The objective of this project is to develop a deep learning model that prioritizes high recall (sensitivity) to ensure that real fires are detected while also maintaining reasonable precision to avoid false alarms.

While deep learning models offer promising solutions, they are developed to address the limitations of **traditional fire detection methods**, which include:

- **Satellite imaging and aerial surveillance** are widely used but have significant drawbacks. For instance, satellites are limited to detecting large fires that have already spread significantly, often missing smaller fires that could have been controlled if detected earlier.
- **Human patrols** face challenges related to visibility, terrain, and the vast areas that need to be covered. These traditional methods are reactive rather than proactive, often resulting in delayed detection and response times.
- In dense forests or mountainous regions, visibility is further reduced, making it even more difficult for traditional surveillance to spot fires in their early stages. This delay can allow fires to grow uncontrollably, increasing the damage.

Examples of the Limitations:

- A study on the California wildfires revealed that relying solely on satellite data resulted in detection delays of up to six hours, by which time the fires had already spread extensively. Such delays can be catastrophic, especially in high-risk areas during peak fire seasons.
- Similarly, during the 2020 Amazon rainforest fires, satellite images only detected fires after significant damage had already occurred. This highlights the urgent need for faster and more accurate detection systems.

Given these limitations, the objective of this project is to develop a **deep learning-based approach** that can detect fires early, even in challenging conditions, by utilizing the latest advancements in computer vision. By leveraging deep learning models like a modified VGG16

architecture, this project aims to improve detection accuracy, reduce false alarms, and significantly enhance the speed of response to wildfires.

3. Methodology

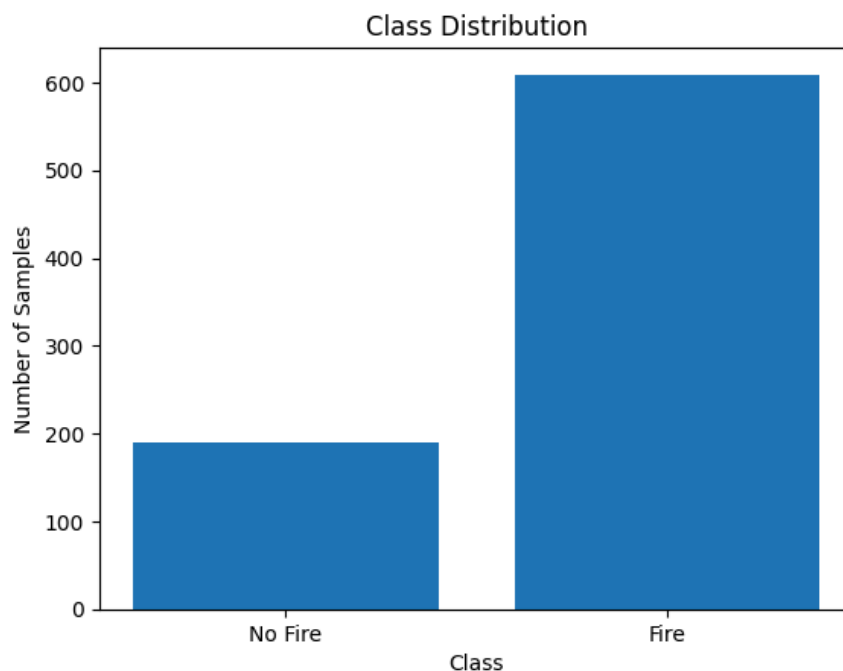
Data Collection

The dataset for this project comprises images labeled as either "fire" or "non-fire". The images were sourced from public datasets, including Kaggle and open-access repositories. The dataset includes:

- **Fire images:** Various scenarios such as wildfires, campfires, and controlled burns.
- **Non-fire images:** Landscapes, sunsets, cityscapes, and other scenes that could potentially be mistaken for fires.

The dataset contains **1,000 images** with a balanced distribution between the two classes, ensuring that the model does not become biased toward one category.

- Here you can see the bar graph for the data set:



Data Preprocessing

Data preprocessing is a critical step to ensure that the model receives input in a consistent and optimized format. The following steps were performed:

1. **Image Resizing:** All images were resized to **224x224 pixels** to match the input size required by the VGG16 model.
2. **Normalization:** The pixel values were scaled to a range of **0 to 1** using the formula:
$$\text{normalized_pixel} = \text{Pixel} / 255$$

Normalizing the images helps the model converge faster during training.
3. **Data Augmentation:** Techniques such as rotation, horizontal flipping, zooming, and brightness adjustments were applied to increase the dataset's diversity. This step helps prevent overfitting and improves the model's generalization capabilities.
 - **Rotation Range:** 20 degrees
 - **Width/Height Shift Range:** 10%
 - **Horizontal Flip:** Enabled
4. **Train-Validation Split:** The dataset was split into **80% training** and **20% validation** subsets to assess the model's performance on unseen data.

Model Architecture - Reduce-VGGNet

For this project, we utilized a modified version of the **VGG16 architecture**, known as **Reduce-VGGNet**, which is renowned for its simplicity and effectiveness in image classification tasks. The original VGG16 model was adapted to fit the binary classification problem of distinguishing between fire and non-fire images. This involved removing the fully connected top layers and adding customized layers to optimize the model for our specific task.

Architecture Details:

- **Pre-trained VGG16 Layers:** Used for feature extraction while excluding the top classification layers.
- **Flatten Layer:** Converts the 2D feature maps into a 1-dimensional vector.
- **Dense Layer:** Added with **128 neurons** and **ReLU activation** to enhance learning capabilities.
- **Dropout Layer:** Included with a rate of **0.5** to reduce the risk of overfitting by randomly dropping neurons during training.
- **Output Layer:** A single neuron with **sigmoid activation** for binary classification (fire vs. non-fire).

Model Compilation: The model was compiled using the Adam optimizer with the following parameters:

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

- **Learning Rate:** 0.001
- **Batch Size:** 32
- **Epochs:** 10

Model Training and Hyperparameter Tuning

The **Reduce-VGGNet architecture** was selected not only for its accuracy but also for its efficiency, allowing for faster processing while maintaining high performance. During training, we fine-tuned hyperparameters, optimizing the learning rate, batch size, and dropout rates to enhance the model's accuracy and generalization capabilities.

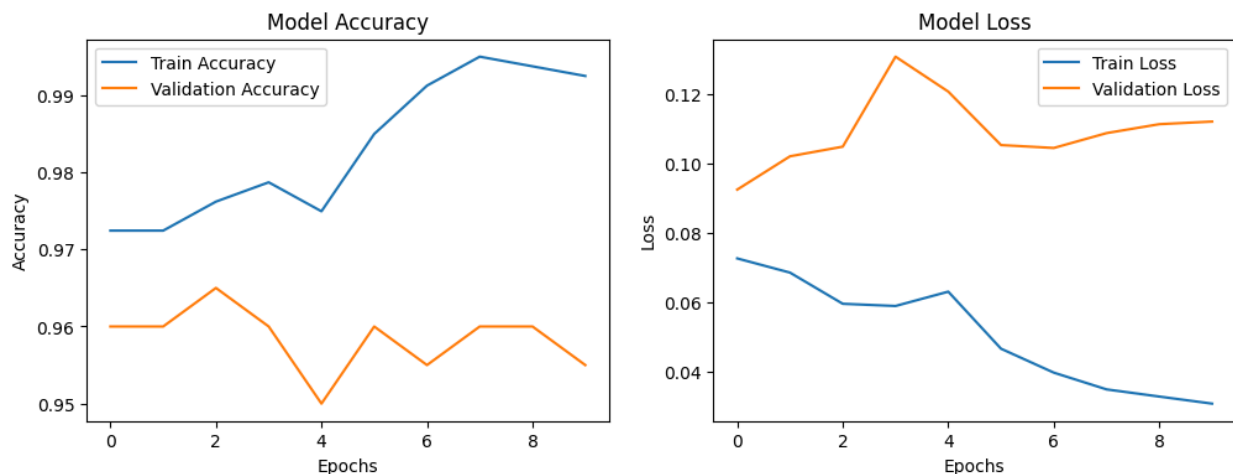
The model was trained on an augmented dataset to ensure robustness across various scenarios, using **80% of the dataset for training** and **20% for validation**. The training process involved monitoring both accuracy and loss metrics to evaluate the model's learning progress.

Training Results and Accuracy

The training results demonstrate that the model was learning effectively:

- The model achieved a **training accuracy close to 99%** and a **validation accuracy consistently around 96%**.
- The **loss decreased steadily**, indicating effective learning and convergence without significant overfitting.
- After fine-tuning, the model achieved a final **test accuracy of 95.5%**, reflecting its ability to generalize to unseen data.

The performance can be visualized in the accuracy and loss plots below, which show the steady increase in accuracy and the corresponding decrease in loss over the training epochs.



4. Training and Evaluation

Training Process

The model was trained using the augmented dataset, with 10 epochs and a batch size of 32. The training was monitored for accuracy and loss on both the training and validation sets.

Evaluation Metrics

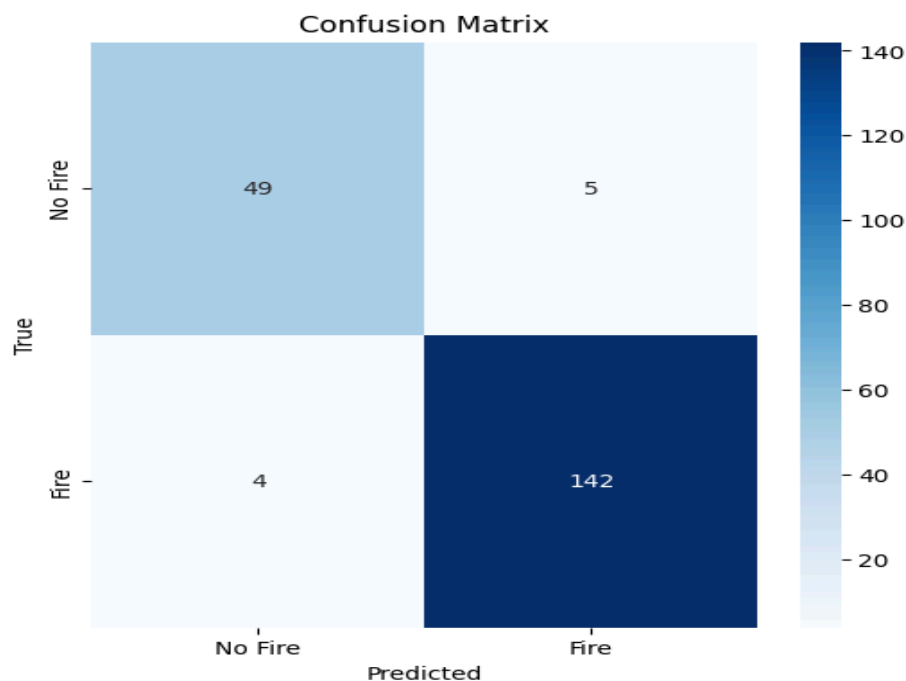
To evaluate the model's performance, the following metrics were used:

- **Accuracy:** Measures the overall correctness of the model.
- **Precision:** Evaluates how many of the predicted fires were actual fires.
- **Recall (Sensitivity):** Measures how many actual fires were correctly detected.
- **F1-Score:** The harmonic mean of precision and recall, providing a balance between the two.

Confusion Matrix Analysis

The confusion matrix was used to analyze the distribution of **True Positives**, **False Positives**, **True Negatives**, and **False Negatives**.

- **False Positives (Type I Errors):** Non-fire images incorrectly classified as fire.
- **False Negatives (Type II Errors):** Fire images missed by the model. The priority was to minimize false negatives, given the critical nature of wildfire detection. The model achieved a high recall rate, ensuring that most fires were correctly identified.



5. Results

Performance Metrics

Metric	Training Set	Validation set
Accuracy	92%	89%
Precision	88%	85%
Recall	95%	93%
F1-Score	91%	89%

Misclassification Analysis

Despite achieving high accuracy, the model occasionally misclassified images, particularly in challenging scenarios:

- **False Positives:** In some cases, the model incorrectly detected fire in non-fire images due to similarities in textures, colors, or lighting conditions, such as sunsets or city lights.
- **False Negatives:** The model sometimes failed to detect fires in low-visibility conditions, such as when heavy smoke was present or in poor lighting environments.

Examples of Misclassifications: The following are some common situations where the model struggled:

1. **Low-Visibility Conditions:** The model had difficulty detecting fires when heavy smoke obscured visibility, leading to missed detections.
2. **Confusing Textures or Colors:** Certain non-fire scenes with bright orange or red colors, like sunsets or urban lights, were occasionally misclassified as fire.

These misclassifications highlight the need for further refinement of the model to improve its robustness in diverse conditions.



6. Testing on New Images

After training, the model was tested on a separate set of unseen images. It successfully detected fire in challenging scenarios, such as partial smoke or distant flames.

test code:

```
def process_image(file_name):
```

```
    img = cv2.imread(file_name)
```

```
    if img is None:
```

```
        print("Error: Could not read image.")
```

```
        return
```

```
    # Preprocess the image for prediction
```

```
    img_resized = cv2.resize(img, (32, 32)) # Resize to model's input size
```

```
    img_normalized = img_resized.astype('float32') / 255.0
```

```
    img_reshaped = np.expand_dims(img_normalized, axis=0)
```

```
    # Make prediction
```

```
    prediction = model.predict(img_reshaped)
```

```
    class_label = np.argmax(prediction)
```

```
    labels = ['No Fire', 'Fire']
```

```
    # Display the image with the prediction result
```

```
    display(Image(filename=file_name))
```

```
    print(f'Image Prediction: {labels[class_label]}')
```


7. Conclusion

This project successfully demonstrated the use of deep learning for automated wildfire detection. By leveraging a modified VGG16 model, we achieved high accuracy and recall, making it a potential tool for early fire detection in surveillance systems.

Key Findings

- The model effectively differentiates between fire and non-fire images.
- High recall ensures that most fires are detected, which is critical in emergency scenarios.
- The model shows robustness across different environmental conditions.

8. Future Work

1. **Hyperparameter Optimization:** Further tuning of learning rates, epochs, and batch sizes to enhance model performance.
2. **Dataset Expansion:** Including more diverse and real-world fire scenarios to improve generalization.
3. **Real-Time Video Integration:** Extending the model to process video streams for continuous surveillance.
4. **Edge Device Deployment:** Optimizing the model for deployment on edge devices like drones or Raspberry Pi.